



Numerical Methods Solver

User's Manual

Rean Glenn A. Roquero

Bachelor of Science in Computer Engineering

Bohol Island State University – Main Campus

Submitted to:

Engr. Bem B. Gumapac, RME, CCpE, MS CpE

CPE 221: Numerical Methods

April 23, 2026

Preface

Welcome to the user manual for the Numerical Methods Solver.

This software was created as the culminating Machine Problem for Module 2 in CPE 221: Numerical Methods, handled by Engr. Bem B. Gumapac. The project applies iterative algorithms and matrix methods discussed in lecture to create a practical solving environment. This manual provides the necessary steps for installation and demonstrates the computational workflow required to achieve accurate numerical solutions.

Table of Contents

Cover Page	
Preface.....	i
Table of Contents	ii
Getting Started	1
Introduction	1
Running the Program	2
Application Interface Tour.....	4
Application Features	6
Function Expression Parsing.....	6
Newton-Raphson Method	7
Gauss-Jacobi Method	10
Newton's Interpolation.....	14
Simpson's 1/3 Rule.....	17
Central Divided Difference + Richardson Extrapolation	20
Source Codes	25
BaseConfigurableSolver.java	25
BaseFunctionSolver.java.....	26
BaseIterativeSolver.java.....	28
SolverConfig.java.....	29
NewtonRaphsonSolver.java	30
GaussJacobiSolver.java.....	32
NewtonInterpolationSolver.java.....	35
SimpsonSolver.java.....	39
CentralDividedDifferenceSolver.java.....	42
References.....	47

Getting Started

1. Introduction

1.1. Project Overview

The  Numerical Methods Solver is a JavaFX desktop application developed to provide an interactive, visual platform for solving common problems in numerical analysis. Unlike command-line scripts or spreadsheet calculations, this solver uses a dedicated graphical user interface (GUI) that allows users to input parameters, visualize iteration tables, and interpret results without needing to write code.

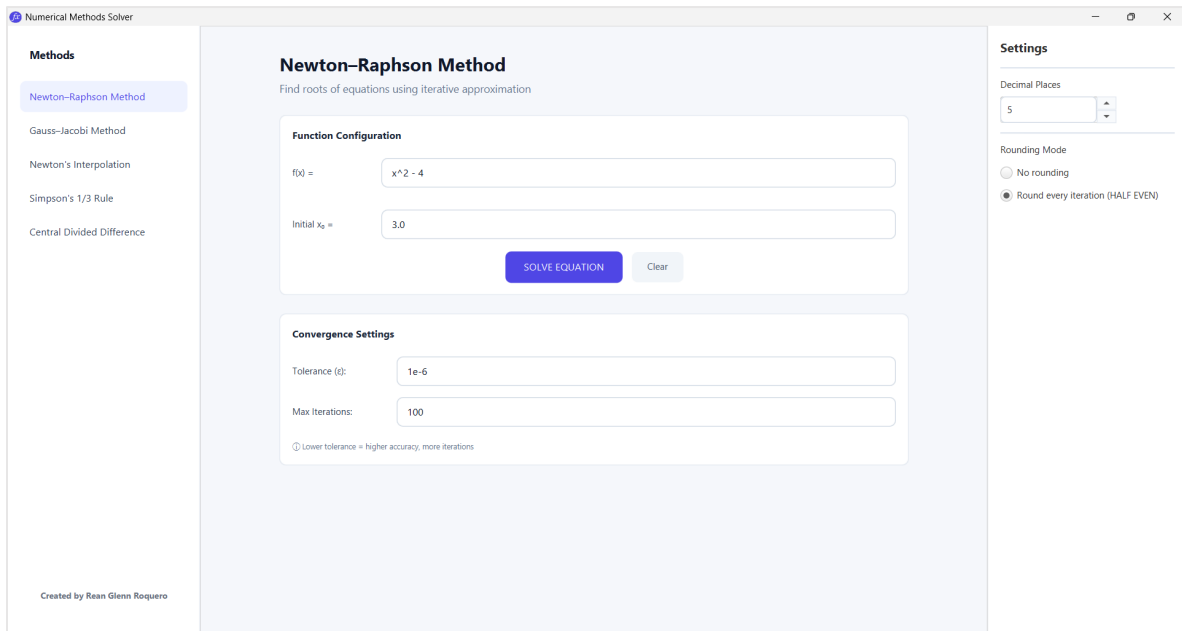


Figure a. Screenshot of Numerical Methods Solver application

This application is focused on five core numerical techniques. These methods were selected to represent a cross-section for root-finding, linear systems, interpolation, integration, and differentiation.

<i>Method</i>	Category	Primary Application
<i>Newton-Raphson</i>	Root-finding	Locating roots of nonlinear equations $f(x) = 0$ using derivative information
<i>Gauss-Jacobi</i>	Linear Systems	Solving systems of equations $Ax = b$ through iterative approximation
<i>Newton's Interpolation</i>	Interpolation	Constructing a polynomial that passes exactly through a given set of data points
<i>Simpson's 1/3 Rule</i>	Numerical Integration	Approximating definite integrals $\int f(x)dx$ using quadratic polynomial fits
<i>Central Divided Difference + Richardson Extrapolation</i>	Numerical Differentiation	Estimating derivatives $f'(x)$ with enhanced accuracy by canceling truncation error

1.2. Technology Stack

The application is built using the following technologies:

-  Java Development Kit (JDK) Version 17+. The core programming language and runtime environment used in this application.
-  Apache Maven Version 4.0.0. A build automation used for this application.
-  JavaFX. An FXML-Based UI framework that provides the overall visual.
-  exp4j Version 0.4.8. A library for thread-safe mathematical expression evaluation and validation.

2. Running the Program

It is best to note that  IntelliJ IDEA Ultimate was used to develop this application.

2.1. Prerequisites

Before attempting to build or run the application, verify that the required tools are installed and accessible from the command line.

- Open a terminal (Command Prompt on Windows, Terminal on macOS/Linux) and execute `java --version`. Expected output should indicate Java 17 or higher. If Java is not installed or the version is older than 17, download and install OpenJDK 17 or Oracle JDK 17 from [Adoptium Eclipse Temurin](#) or [Oracle JDK Archive](#).
- To verify Maven installation, execute `mvn -version`. Expected output should indicate Maven 3.6 or higher, Otherwise, download and install it from [Apache Maven Download Page](#).
- Navigate to the root directory of the project and confirm that the `pom.xml` exists and other source files too.

2.2. Run the Application

Once the prerequisites are all done, the recommended method for running it is the JavaFX Maven Plugin execution command `mvn javafx:run`.

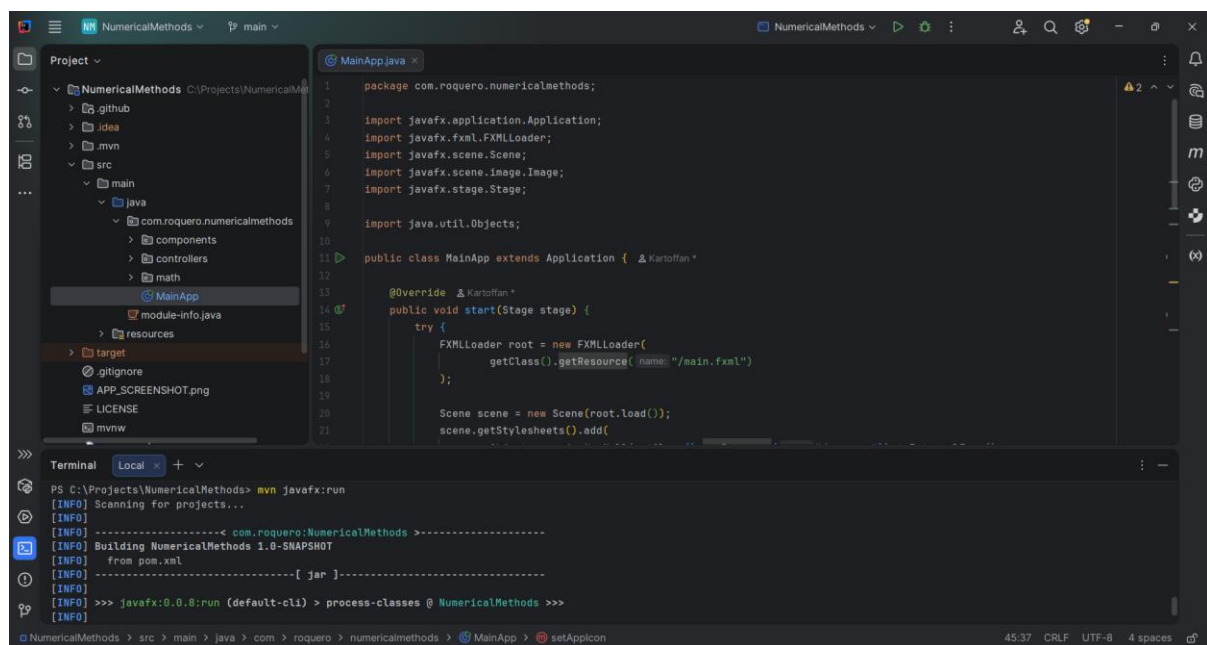


Figure b. Screenshot of the project in IntelliJ IDEA Ultimate

3. Application Interface Tour

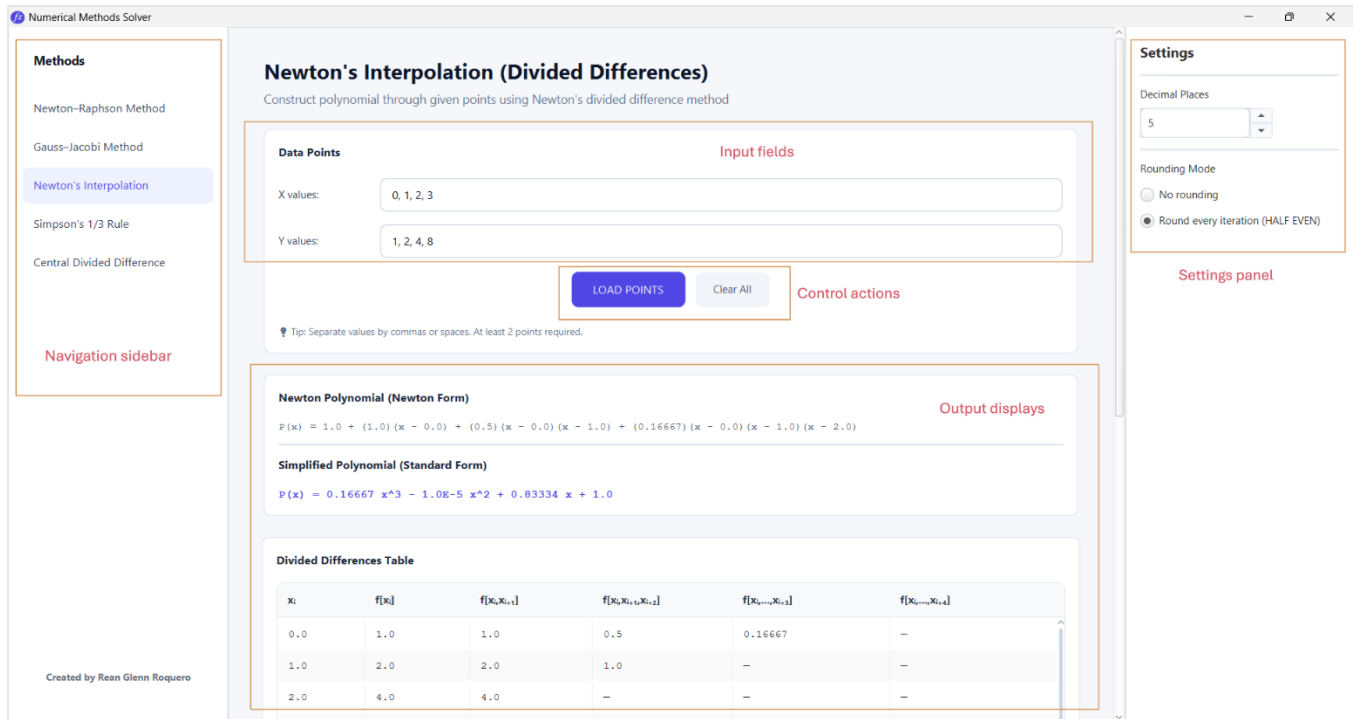


Figure c. Layout names of Numerical Methods Solver

3.1. Navigation Sidebar

On the left side of the application window, a vertical navigation panel presents the complete list of available numerical methods where you can quickly switch between methods. This is the primary entry point to the system. Each of the five numerical methods is displayed as a selectable option:

- Newton-Raphson Method
- Gauss-Jacobi Method
- Newton's Interpolation
- Simpsons 1/3 Rule
- Central Divided Difference

3.2. Workspace Area

This is the focal point of your interaction. When a method is chosen from the left sidebar, its corresponding interface is dynamically rendered. Each method presents a set of components:

- Input fields
- Action controls
- Output displays

3.3. Settings Panel

This is the centralized control over numerical precision and rounding behavior. The first setting allows you to specify the number of decimal places displayed in all numerical outputs. The chosen value determines how many digits appear after the decimal point in results such as root approximates and integral values. For example, a setting of 4 will display the value 3.1416 rather than 3.1415926535.

On the other hand, the second setting determines how and when rounding is applied during computation.

- No rounding – Displays raw floating-point values exactly as computed by the Java runtime. Results may include many decimal places.
- Round every iteration - Applies rounding at each computational step using the HALF_EVEN convention (also known as Banker's Rounding or Odd-Even Rule).

Application Features

1. Function Expression Parsing

Numerical Methods Solver relies on a robust mathematical expression parsing engine to evaluate user-provided functions across all five method modules. This capability is powered by exp4j (Expression for Java), a lightweight, open-source library that interprets string-based mathematical expressions and computes their value at specified points. Understanding the syntax rules and capabilities of the expression parser is essential for successful use of the application.

- a. The expression parser recognizes the following standard arithmetic operators:

Operator	Symbol	Description	Example
Addition	+	Adds two values.	$2 + 3 \rightarrow 5$
Subtraction	-	Subtracts right operand from left.	$5 - 2 \rightarrow 3$
Multiplication	*	Multiplies two values	$3 * 4 \rightarrow 12$
Division	/	Divides left operand by right.	$10 / 2 \rightarrow 5$
Exponentiation	^	Raises left operand to right power.	$2^3 \rightarrow 8$
Modulo	%	Remainder of division	$7 \% 3 \rightarrow 1$
Unary Minus	-	Negates a value	$-5 \rightarrow -5$
Unary Plus	+	Positive sign (no effect)	$+5 \rightarrow 5$

- b. The parser includes a comprehensive set of mathematical functions.

Function	Description	Example
<code>sin(x)</code>	Sine of x (radians)	<code>sin(1.5708) → 1.0</code>
<code>cos(x)</code>	Cosine of x (radians)	<code>cos(0) → 1.0</code>
<code>tan(x)</code>	Tangent of x (radians)	<code>tan(0.7854) → 1.0</code>
<code>asin(x)</code>	Arc sine (inverse sine)	<code>asin(0) → 0.0</code>
<code>acos(x)</code>	Arc cosine (inverse cosine)	<code>acos(1) → 0.0</code>
<code>atan(x)</code>	Arc tangent (inverse tangent)	<code>atan(1) → 0.7854</code>
<code>sinh(x)</code>	Hyperbolic sine	<code>sinh(0) → 0.0</code>
<code>cosh(x)</code>	Hyperbolic cosine	<code>cosh(0) → 1.0</code>
<code>tanh(x)</code>	Hyperbolic tangent	<code>tanh(0) → 0.0</code>
<code>exp(x)</code>	e raised to power x	<code>exp(1) → 2.71828</code>
<code>log(x)</code>	Natural logarithm (base e)	<code>log(2.71828) → 1.0</code>
<code>log10(x)</code>	Logarithm base 10	<code>log10(100) → 2.0</code>
<code>log2(x)</code>	Logarithm base 2	<code>log2(8) → 3.0</code>
<code>sqrt(x)</code>	Square root	<code>sqrt(16) → 4.0</code>
<code>cbrt(x)</code>	Cube root	<code>cbrt(27) → 3.0</code>
<code>pow(x,y)</code>	x raised to power y	<code>pow(2, 3) → 8.0</code>
<code>abs(x)</code>	Absolute value	<code>abs(-5) → 5.0</code>

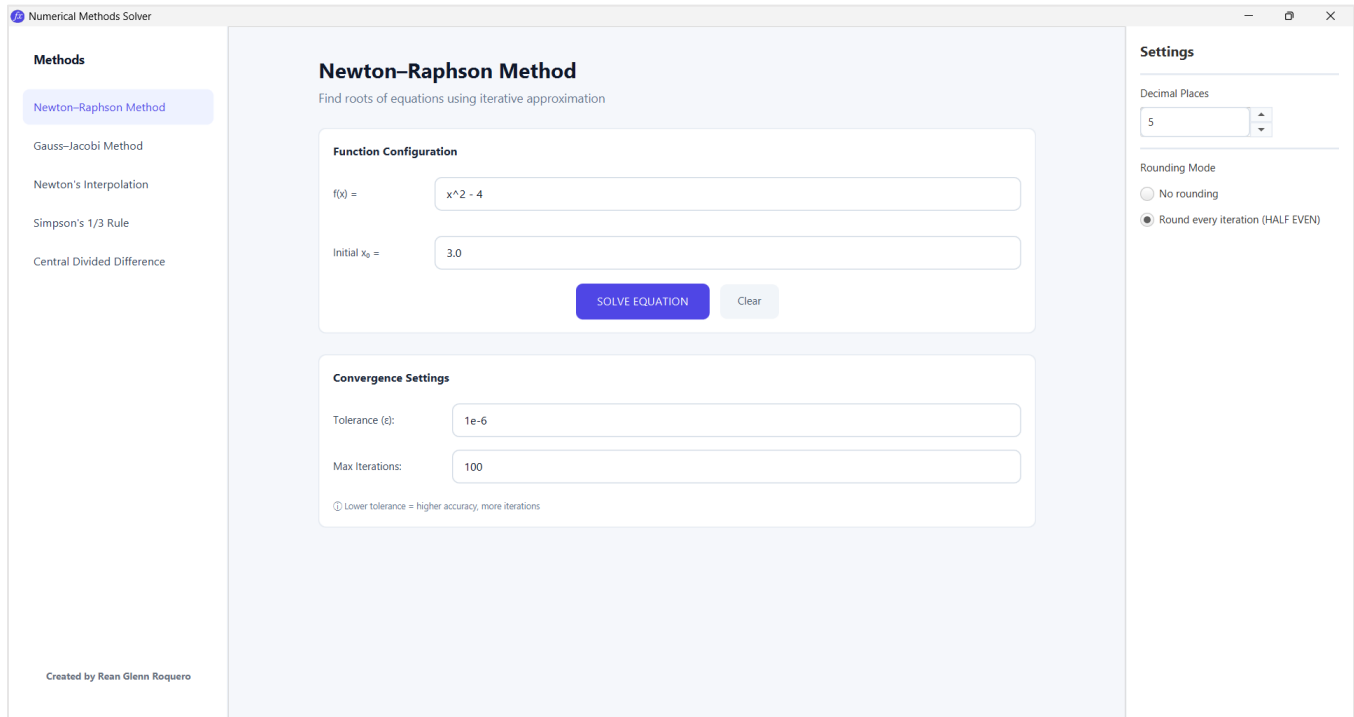
ceil(x)	Smallest integer $\geq x$	ceil(3.2) $\rightarrow$ 4.0
floor(x)	Largest integer $\leq x$	floor(3.8) $\rightarrow$ 3.0
signum(x)	Sign function (-1, 0, or 1)	signum(-7) $\rightarrow$ -1.0

- c. The parser automatically recognizes the following mathematical constants:

Constant	Symbol(s)	Value
Pi	pi , π	3.141592653589793
Euler's number	e	2.718281828459045
Golden ratio	ϕ	1.61803398874

- d. The parser supports scientific notation for entering very large or very small numbers using the **e** or **E** exponent indicator. For example, **1.5e3 $\rightarrow$ 1500.0**.
- e. The parser supports implicit multiplication allowing users to omit the ***** operator in many common cases. For example, instead of typing **2*x**, the parser allows it to be just **2x**.

2. Newton-Raphson Method



The screenshot shows the 'Numerical Methods Solver' application window. On the left is a sidebar with 'Methods' listed: Newton-Raphson Method (selected), Gauss-Jacobi Method, Newton's Interpolation, Simpson's 1/3 Rule, and Central Divided Difference. The main area is titled 'Newton-Raphson Method' with the subtitle 'Find roots of equations using iterative approximation'. It contains two sections: 'Function Configuration' and 'Convergence Settings'. In 'Function Configuration', 'f(x) =' is set to 'x^2 - 4' and 'Initial x₀ =' is set to '3.0'. There are 'SOLVE EQUATION' and 'Clear' buttons. In 'Convergence Settings', 'Tolerance (ε):' is set to '1e-6' and 'Max Iterations:' is set to '100'. A note at the bottom says 'Lower tolerance = higher accuracy, more iterations'. On the right is a 'Settings' panel with 'Decimal Places' set to '5' and 'Rounding Mode' set to 'Round every iteration (HALF EVEN)'.

The Newton-Raphson method is an open root-finding algorithm that uses the derivative of a function to iteratively converge toward a root. Unlike bracketing methods, it does not require an interval that contains the root. Instead, it relies on an initial guess and the local behavior of the function to produce successively better approximations.

2.1. Mathematical Background

The Newton-Raphson iteration formula is defined as

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

where x_n is the current approximation of the root, x_{n+1} is the next (improved) approximation, $f(x_n)$ is the function value at x_n and $f'(x_n)$ is the derivative value at x_n .

The method exhibits quadratic convergence when the initial guess is sufficiently close to the true root and the derivative at the root is non-zero. However, the method may diverge or fail entirely if:

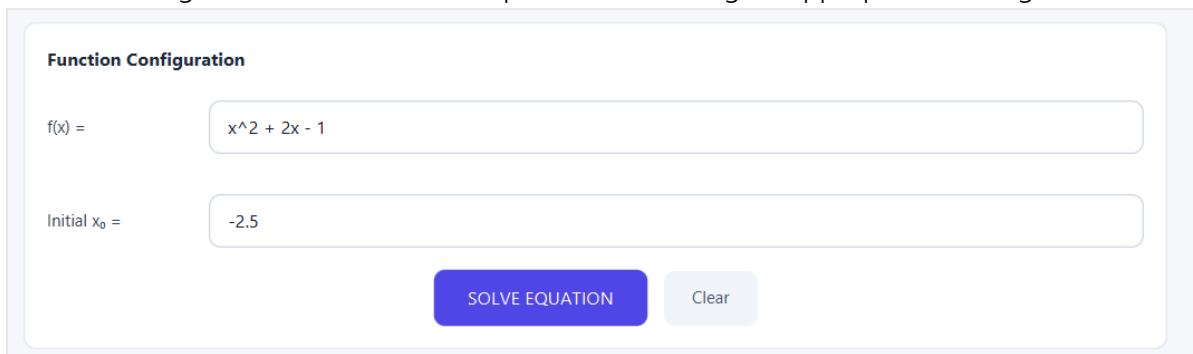
- The initial guess is too far from the true root.
- The derivative $f'(x_n)$ approaches zero causing division by a very small number.
- The function has inflection points or local extrema near the initial guess.

These potential failure modes are directly addressed in the Numerical Methods Solver through input validation and warning messages.

2.2. Accessing the Module

The Newton-Raphson module is the default view upon application launch. It can also be accessed at any time by clicking the “Newton-Raphson Method” button in the left navigation sidebar.

- Type the mathematical expression for $f(x)$. Ensure that the syntax follows the supported conventions.
- Enter a starting value into the second input field. Choosing an appropriate initial guess is critical.



Function Configuration

f(x) =

Initial x_0 =

- Adjust the tolerance ε and maximum iteration fields as needed.

Convergence Settings

Tolerance (ε):

Max Iterations:

ⓘ Lower tolerance = higher accuracy, more iterations

- d. Click the “SOLVE EQUATION” button. The application will perform the iterative calculation. After that, you will get the results and the table of iterations.

Solution

Root: **-2.41421**

Converged: **✓ Yes**

Convergence Test: 0.05556

Iterations: 4

Final Error: 0.0

Newton-Raphson Formula

f Iteration Formula

$$x_{n+1} = x_n - f(x_n) / f'(x_n)$$

Iteration Details

n	x_n	$f(x)$	$f'(x)$	x_{n+1}	Error
1	-2.5	0.25	-3.0	-2.41667	0.08333
2	-2.41667	0.00695	-2.83334	-2.41422	0.00245
3	-2.41422	2.0E-5	-2.82844	-2.41421	1.0E-5
4	-2.41421	-1.0E-5	-2.82842	-2.41421	0.0

2.3. Error Handling and Troubleshooting

Symptom	Possible Cause	Resolution
"Function $f(x)$ cannot be empty"	Function field is blank.	Enter a valid mathematical function.
"Invalid mathematical expression"	Syntax error in function string.	Verify supported function names and unbalanced parentheses.
Negative status with empty iteration table	Expression evaluation failed on first iteration.	Choose an initial guess within the function's domain.
Negative status with a few iterations	Derivative became near zero.	Select a different initial guess away from stationary points.
Negative status with full iteration count	Slow convergence or poor initial guess	Increase max iterations, loosen tolerance, or change x_0 .
Convergence test value > 1	Initial guess is far from root or near inflection.	Select a different x_0 and observe new test value.
Root value is "Failed."	Method terminated without convergence.	Apply appropriate remedy based on failure mode.
Application freezes during computation.	Very large iteration count with complex function.	Reduce max iterations or use a simpler test function.

3. Gauss-Jacobi Method

Numerical Methods Solver

Methods

- Newton-Raphson Method
- Gauss-Jacobi Method**
- Newton's Interpolation
- Simpson's 1/3 Rule
- Central Divided Difference

Created by Rean Glenn Roquero

Gauss-Jacobi Method

Solve systems of linear equations using iterative approximation

System Configuration

Matrix Size (n×n): Generate Matrix

Convergence Settings

Tolerance (ε):

Max Iterations:

① Lower tolerance = higher accuracy, more iterations

Settings

Decimal Places:

Rounding Mode:

☐ No rounding

☒ Round every iteration (HALF EVEN)

The Gauss-Jacobi method is an iterative algorithm for solving systems of linear equations of the form $\mathbf{Ax} = \mathbf{b}$. Unlike direct methods such as Gaussian Elimination, which produce an exact solution in a finite number of steps, Jacobi generates a sequence of approximations that converge toward the true solution under certain conditions. This module implements the classical Jacobi iteration with a practical limitation of up to five variables.

3.1. Mathematical Background

Given a system of n linear equations with n unknowns:

$$\begin{aligned} a_{11}x_1^{(1)} + a_{12}x_2^{(1)} + \cdots + a_{1n}x_n^{(1)} &= b_1 \\ a_{21}x_1^{(2)} + a_{22}x_2^{(2)} + \cdots + a_{2n}x_n^{(2)} &= b_2 \\ &\vdots \\ a_{n1}x_1^{(n)} + a_{n2}x_2^{(n)} + \cdots + a_{nn}x_n^{(n)} &= b_n \end{aligned}$$

or in its linear algebraic form $\mathbf{Ax} = \mathbf{b}$, where $\mathbf{A}$ is the $n \times n$ coefficient matrix, $\mathbf{x}$ is the solution vector, and $\mathbf{b}$ is the constants vector of length n .

The Jacobi method solves for each variable independently using values from the previous iteration. For each $i = 1, 2, \dots, n$, the compact form is defined as

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right)$$

Convergence Condition:

The Jacobi method is guaranteed to converge if the coefficient matrix $\mathbf{A}$ is strictly diagonally dominant. A matrix is strictly diagonally dominant when, for every row, the absolute value of the diagonal element is greater than the sum of the absolute values of all other elements in that row

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|$$

for all $i = 1, 2, \dots, n$.

In this application, it automatically rearranges the rows of the coefficient matrix and constants vector to satisfy this convergence condition. It, however, returns an error when there's no possible permutation satisfying the condition.

3.2. Accessing the Module

The Gauss-Jacobi module is accessed by clicking the "Gauss-Jacobi Method" button in the left navigation sidebar. The module supports systems of 2, 3, 4, or 5 variables. This limitation ensures that

the matrix input interface remains manageable and visually clear and computation time remains instantaneous.

- a. Enter the matrix size, that is, the number of variables. The matrix input grid will show up as you click the “Generate Matrix” button.

System Configuration

Matrix Size (n×n): Generate Matrix

- b. Fill each cell of the $n \times n$ grid with appropriate coefficient values. Ensure that diagonal elements a_{ii} are non-zero as this will cause division by zero during iteration. To satisfy the convergence condition, the application automatically rearranges the rows to satisfy it if possible.
- c. Fill the column for the constants, that is, with terms $b_1, b_2, \dots, b_n$.
- d. Optionally, provide an initial guess found at the rightmost of the grid. If left blank, the zero vector $[0, 0, \dots, 0]$ is used.

System Definition

A · x = b Initial guess: x₀

	x1 column 1	x2 column 2	x3 column 3	b constants	x ₀ initial guess
Eq 1 row 1	2	5	1	4	1
Eq 2 row 2	1	2	6	1	1
Eq 3 row 3	8	1	2	-1	1

Tip: The system will be automatically reordered for diagonal dominance to ensure convergence

- e. Adjust the tolerance ε and the maximum iterations as needed.

Convergence Settings

Tolerance (ϵ):

Max Iterations:

ⓘ Lower tolerance = higher accuracy, more iterations

- f. Execute the method by clicking "Solve System." The application will perform the iterative calculation and display results.

Solution

Solution Vector: **$x_1 = -0.21327$ $x_2 = 0.90521$ $x_3 = -0.09952$**

Converged: **✓ Yes**

Iterations: 19

Final Error: 0.0

Iteration Details						
Iteration	x_1	x_2	x_3	x_4	x_5	Maximum Error
1	-0.5	0.2	-0.33333	—	—	1.5
2	-0.06667	1.06667	0.18333	—	—	0.86667
3	-0.30417	0.79	-0.17778	—	—	0.36111
4	-0.1793	0.95722	-0.04597	—	—	0.16722
5	-0.23316	0.88091	-0.12252	—	—	0.07655
6	-0.20448	0.91777	-0.08811	—	—	0.03686
7	-0.21769	0.89941	-0.10518	—	—	0.01836
8	-0.21113	0.90811	-0.09686	—	—	0.0087
9	-0.2143	0.90382	-0.10085	—	—	0.00429
10	-0.21276	0.90589	-0.09889	—	—	0.00207
11	-0.21351	0.90488	-0.09984	—	—	0.00101
12	-0.21315	0.90537	-0.09937	—	—	4.9E-4
13	-0.21333	0.90513	-0.0996	—	—	2.4E-4
14	-0.21324	0.90525	-0.09949	—	—	1.2E-4
15	-0.21328	0.90519	-0.09954	—	—	6.0E-5
16	-0.21326	0.90522	-0.09952	—	—	3.0E-5
17	-0.21327	0.90521	-0.09953	—	—	1.0E-5
18	-0.21327	0.90521	-0.09952	—	—	1.0E-5
19	-0.21327	0.90521	-0.09952	—	—	0.0

3.3. Error Handling and Troubleshooting

Symptom	Possible Cause	Resolution
"Please enter a valid matrix size (2-5)."	Non-numeric matrix size	Enter an integer value between 2 and 5.
"Matrix size must be at least 2."	Matrix size less than 2	Enter a value of 2 or greater.
"Maximum matrix size is 5 for optimal display."	Matrix size greater than 5	Enter a value of 5 or less, or accept the automatic adjustment.
"Please generate the matrix input fields first."	Matrix not yet generated	Click "Generate Matrix: before attempting to solve.
"Please enter valid numeric values."	Non-numeric entries	Replace any letters or invalid symbols with numeric values.
"Zero on diagonal at row X"	Zero on diagonal	Input non-zero on diagonal entries.
"No diagonally dominant arrangement found."	No diagonally dominant arrangement found	Abandon the current system and use another.

4. Newton's Interpolation

Numerical Methods Solver

Methods

- Newton-Raphson Method
- Gauss-Jacobi Method
- Newton's Interpolation**
- Simpson's 1/3 Rule
- Central Divided Difference

Created by Rean Glenn Roquero

Newton's Interpolation (Divided Differences)

Construct polynomial through given points using Newton's divided difference method

Data Points

X values:

Y values:

Tip: Separate values by commas or spaces. At least 2 points required.

Settings

Decimal Places:

Rounding Mode:

☐ No rounding

☒ Round every iteration (HALF EVEN)

Newton's Interpolation is a polynomial interpolation method that constructs a unique polynomial of degree $n - 1$ that passes exactly through a given set of n data points. Newton's formulation uses divided difference to build the polynomial incrementally making it particularly efficient when adding new data points or evaluating at multiple values.

4.1. Mathematical Background

Given a set of n data points $(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})$ with distinct x values, Newton's interpolation formula expresses the interpolating polynomial as

$$P(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) \\ + \dots + f[x_0, x_1, \dots, x_{n-1}](x - x_0)(x - x_1) \dots (x - x_{n-2})$$

where $f[x_i, x_{i+1}, \dots, x_j]$ represents the divided difference of order $j - i$.

First-Order Divided Difference:

$$f[x_i, x_j] = \frac{f(x_j) - f(x_i)}{x_j - x_i}$$

Higher-Order Divided Difference:

$$f[x_i, x_{i+1}, \dots, x_j] = \frac{f[x_{i+1}, \dots, x_j] - f[x_i, \dots, x_{j-1}]}{x_j - x_i}$$

4.2. Accessing the Module

The Newton's Interpolation module is accessed by clicking the "Newton's Interpolation" button in the left navigation sidebar.

- We only need the set of ordered pairs by inputting the x and y values in their respective fields.

Data Points

X values:

Y values:

LOAD POINTS

Clear All

💡 Tip: Separate values by commas or spaces. At least 2 points required.

- b. Click the "LOAD POINTS" button and observe the result display.

Newton Polynomial (Newton Form)

$$P(x) = 1.0 + (1.0)(x - 0.0) + (0.5)(x - 0.0)(x - 1.0) + (0.16667)(x - 0.0)(x - 1.0)(x - 2.0)$$

Simplified Polynomial (Standard Form)

$$P(x) = 0.16667 x^3 - 1.0E-5 x^2 + 0.83334 x + 1.0$$

Divided Differences Table

x_i	$f[x_i]$	$f[x_i, x_{i+1}]$	$f[x_i, x_{i+1}, x_{i+2}]$	$f[x_i, \dots, x_{i+3}]$	$f[x_i, \dots, x_{i+4}]$
0.0	1.0	1.0	0.5	0.16667	—
1.0	2.0	2.0	1.0	—	—
2.0	4.0	4.0	—	—	—
3.0	8.0	—	—	—	—

- c. Input different values of x and click "INTERPOLATE" to obtain the interpolated values of y .

Interpolation

X value(s): INTERPOLATE

Interpolation Results:

$f(1.0) = 2.0$
 $f(1.5) = 2.8125$
 $f(2.0) = 4.0$
 $f(5.0) = 26.0002$
 $f(6.0) = 42.0004$

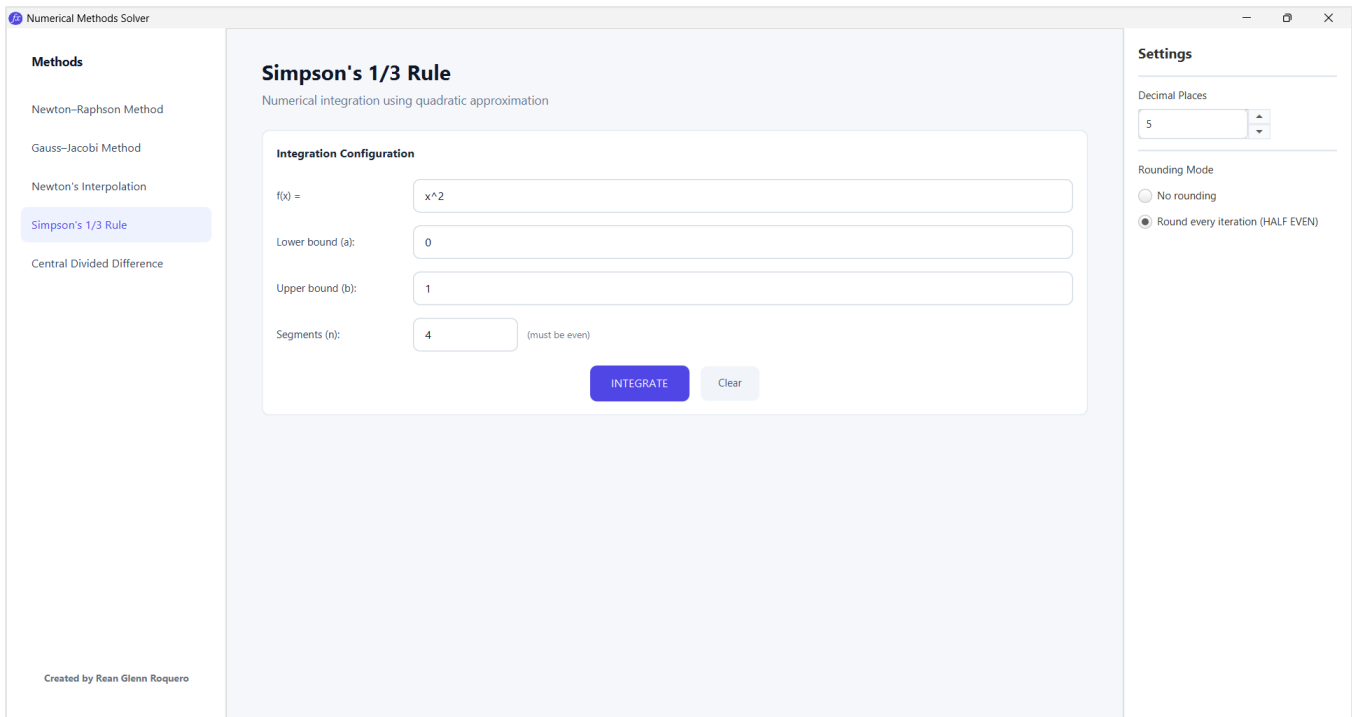
Interpolation Table

X Value	Interpolated Y
1.0	2.0
1.5	2.8125
2.0	4.0
5.0	26.0002
6.0	42.0004

4.3. Error Handling and Troubleshooting

Symptom	Possible Cause	Resolution
Divided difference table shows all zeros.	y values are all identical.	This is correct behavior for constant function; polynomial will be constant.
Divided difference table shows NaN.	Duplicate x values or numerical overflow.	Ensure all x values are distinct; use moderate x values.
Polynomial display is truncated or overflowing.	Long polynomial strings exceeds label width.	Label wraps text automatically; resize window if needed.
Interpolation returns unexpected value.	Input x outside data range.	Extrapolation is valid but may be inaccurate.
Interpolation field accepts multiple values but only last shown.	Multiple x values entered but result label overwrites.	Multiple values are computed but only last may be visible in label; check interpolation table for all results

5. Simpson's 1/3 Rule



The screenshot shows the 'Numerical Methods Solver' application window. On the left is a 'Methods' sidebar with options: Newton-Raphson Method, Gauss-Jacobi Method, Newton's Interpolation, Simpson's 1/3 Rule (selected), and Central Divided Difference. The main area is titled 'Simpson's 1/3 Rule' with the subtitle 'Numerical integration using quadratic approximation'. It contains an 'Integration Configuration' section with input fields for:

- $f(x) =$: x^2
- Lower bound (a): 0
- Upper bound (b): 1
- Segments (n): 4 (with a note '(must be even)')

 Below these fields are 'INTEGRATE' and 'Clear' buttons. On the right is a 'Settings' panel with 'Decimal Places' set to 5 and 'Rounding Mode' set to 'Round every iteration (HALF EVEN)'.

Simpson's 1/3 Rule is a numerical integration technique that approximates the definite integral of a function over a closed interval. It belongs to the family of Newton-Cotes quadrature formulas and improves upon simpler methods like Trapezoidal Rule by fitting quadratic polynomial to successive pairs of

subintervals. This module provides an interactive interface for evaluating definite integrals with user-configurable precision through segment count control.

5.1. Mathematical Background

Given a function $f(x)$ defined on the interval $[a, b]$, Simpson's 1/3 Rule approximates the definite integral by dividing the interval into n equal segments (where n must be even) and summing the areas under quadratic interpolating polynomials.

$$\int_a^b f(x) dx \approx \frac{h}{3} \left[f(x_0) + 4 \sum f(x_{odd}) + 2 \sum f(x_{even}) + f(x_n) \right]$$

where $h = \frac{b-a}{n}$, $x_{odd} = x_1, x_3, x_5, \dots$, and $x_{even} = x_2, x_4, x_6, \dots$

The error in Simpson's 1/3 Rule is proportional to the fourth derivative of the function

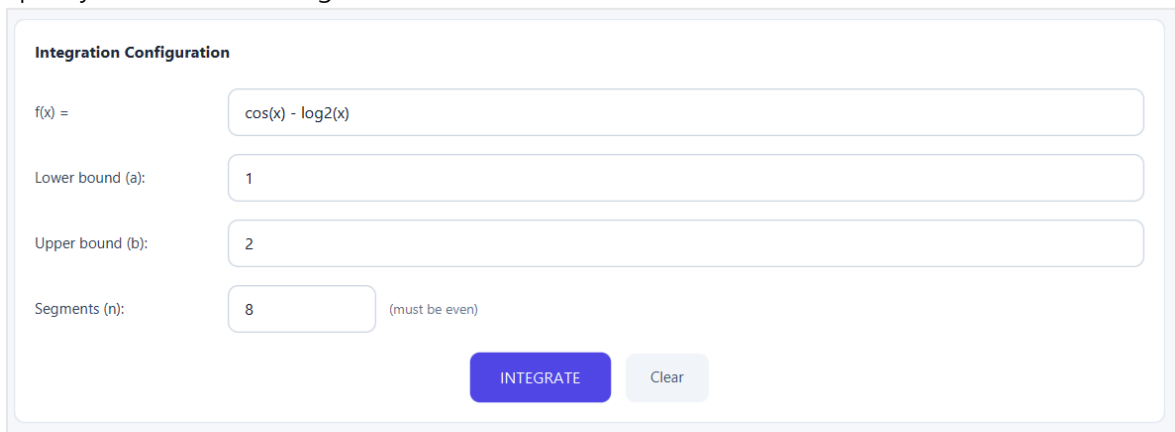
$$Error = -\frac{(b-a)^5}{180n^4} \cdot f^{(4)}(\xi)$$

for some $\xi \in [a, b]$.

5.2. Accessing the Module

The Simpson's 1/3 Rule module is accessed by clicking the "Simpson's 1/3 Rule" button in the left navigation sidebar.

- Type the mathematical expression for $f(x)$, Ensure the syntax follows the supported conventions.
- Enter the lower and upper bounds of integration.
- Specify the number of segments n .



Integration Configuration

f(x) =

Lower bound (a):

Upper bound (b):

Segments (n): (must be even)

- Click "INTEGRATE" button to display the results.

Integration Result

Integral Value: **-0.48948**

Segments Used: 8

Step Size (h): 0.125

Status: **✓ Complete**

Detailed Solution Steps

Simpson's 1/3 Rule Formula

∫ General Formula

$$\int_a^b f(x) dx \approx (h/3) [f(x_0) + f(x_n) + 4\sum f(x_{\text{odd}}) + 2\sum f(x_{\text{even}})]$$

$$\text{Applied: } I = (0.125/3) \times -11.74741$$

Calculation Summary

Σ Weighted Sum

$$\sum c_i \cdot f(x_i) = -11.74741$$

✓ Final Result

$$I = -0.48948$$

Point-by-Point Calculation

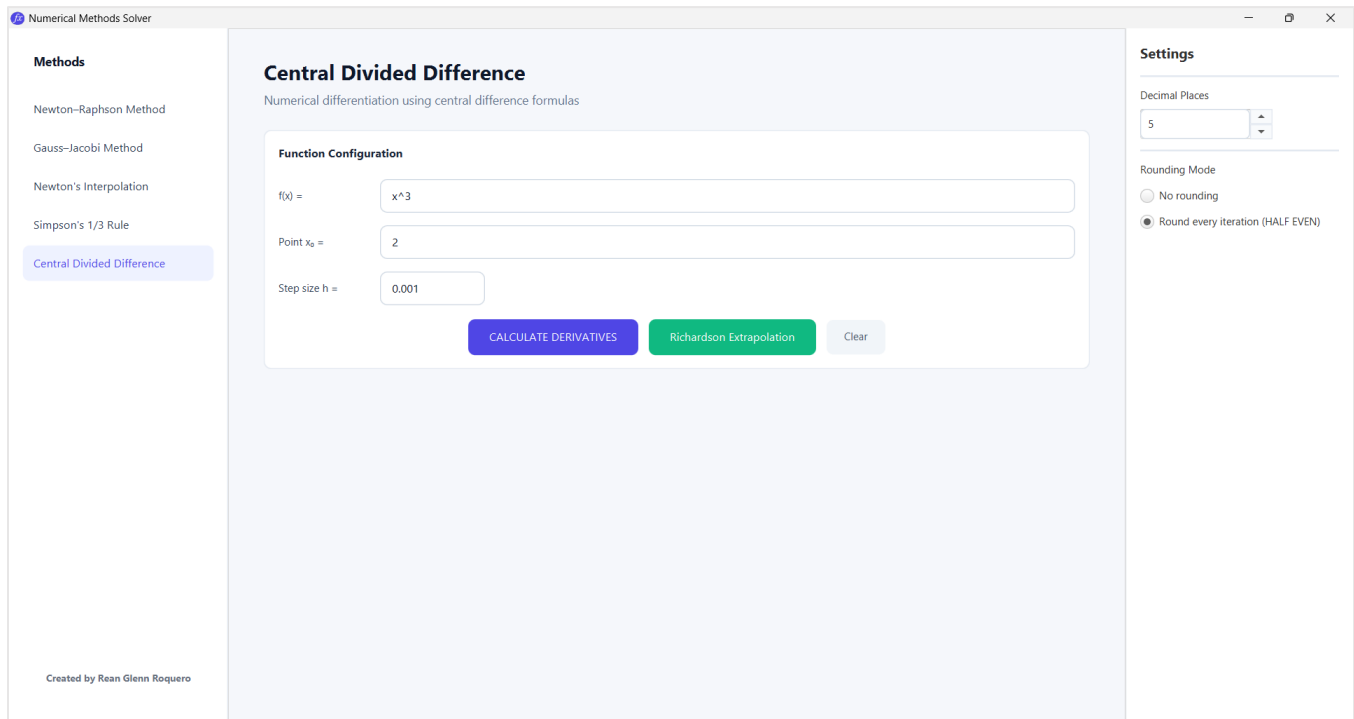
Coefficients: 1 = endpoints, 2 = even indices, 4 = odd indices

i	x_i	$f(x_i)$	Coefficient	$c_i \cdot f(x_i)$
0	1.0	0.5403	1 (endpoint)	0.5403
1	1.125	0.26125	4 (odd)	1.045
2	1.25	-0.00661	2 (even)	-0.01322
3	1.375	-0.26488	4 (odd)	-1.05952
4	1.5	-0.51423	2 (even)	-1.02846
5	1.625	-0.75462	4 (odd)	-3.01848
6	1.75	-0.9856	2 (even)	-1.9712
7	1.875	-1.20642	4 (odd)	-4.82568
8	2.0	-1.41615	1 (endpoint)	-1.41615
			Σ =	-11.74741

5.3. Error Handling and Troubleshooting

Symptom	Possible Cause	Resolution
"Lower bound must be less than upper bound"	$a \geq b$	Enter $a < b$.
"Number of segments must be at least 2"	$n < 2$ or $n > 1000$	Enter a value between 2 and 1000.
"Number of segments must be EVEN"	n is odd	Enter an even number: 2, 4, 6, 8, etc.
Result is NaN or Infinity	Function has singularity in $[a, b]$.	Simpson's Rule cannot handle singularities; adjust interval.

6. Central Divided Difference + Richardson Extrapolation



The screenshot shows the 'Numerical Methods Solver' application window. On the left, a sidebar lists various methods, with 'Central Divided Difference' selected. The main area is titled 'Central Divided Difference' and includes a subtitle 'Numerical differentiation using central difference formulas'. Below this is a 'Function Configuration' section with three input fields: 'f(x) =' with the value 'x^3', 'Point x_0 =' with the value '2', and 'Step size h =' with the value '0.001'. At the bottom of this section are three buttons: 'CALCULATE DERIVATIVES' (blue), 'Richardson Extrapolation' (green), and 'Clear' (grey). On the right side of the window is a 'Settings' panel with 'Decimal Places' set to 5 and 'Rounding Mode' set to 'Round every iteration (HALF EVEN)'.

The Central Divided Difference (CDD) method combined with Richardson Extrapolation is a numerical differentiation technique that estimates the derivative of a function at a given point with enhanced accuracy. While the basic CDD formula provides an $O(h^2)$ approximation, Richardson Extrapolation combines results from two different step sizes to cancel the leading error term, yielding an $O(h^4)$ estimate. This module demonstrates the power of error cancellation techniques in numerical analysis.

6.1. Mathematical Background

Numerical differentiation approximates the derivative $f'(x)$ using finite difference formulas. The choice of formula and step size significantly affects accuracy. Below is the formula for Central Divided Difference

$$f'(x) = \frac{f(x+h) - f(x-h)}{2h}$$

$$f''(x) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$$

The Richardson extrapolated value is

$$D' = \frac{4D\left(\frac{h}{2}\right) - D(h)}{3}$$

$$D'' = \frac{16D\left(\frac{h}{2}\right) - D(h)}{15}$$

6.2. Accessing the Module

The Central Divided Difference module is accessed by clicking the "Central Divided Difference" button in the left navigation sidebar.

- Type the mathematical expression for $f(x)$.
- Enter the x_0 value at which to compute the derivative.
- Enter a step size h . A value of 0.1 is a reasonable default for many functions.

Function Configuration

f(x) =

Point x_0 =

Step size h =

d. Click the "CALCULATE DERIVATIVES" to display the results.

Numerical Results

First Derivative $f'(x)$: **0.49915**

Second Derivative $f''(x)$: **-0.49875**

Truncation Error Estimate: 2.1E-4

Detailed Solution Steps

First Derivative $f'(x)$

► **Formula**

$$f'(x) = [f(x+h) - f(x-h)] / (2h)$$

► **Calculation**

$$f'(0.0) = [f(0.1) - f(-0.1)] / (2 \times 0.1) = [0.04742 - -0.05241] / 0.2 = 0.49915$$

✓ **Result**

0.49915

Second Derivative $f''(x)$

► **Formula**

$$f''(x) = [f(x+h) - 2f(x) + f(x-h)] / h^2$$

► **Calculation**

$$f''(0.0) = [f(0.1) - 2f(0.0) + f(-0.1)] / 0.1^2 = [0.04742 - 2 \times 0.0 + -0.05241] / 0.010000000000000002 = -0.49875$$

✓ **Result**

-0.49875

e. Click "Richardson Extrapolation" to show the extrapolated value.

Richardson Extrapolation

Extrapolated Results

Derivative	Step Size (h)	Value	Error
$f'(x)$	0.0125	0.5	0.0
$f''(x)$	0.0125	-0.5	0.0

Step-by-Step Solution

First Derivative

Initial Approximation

$$D(h) = [f(x+h) - f(x-h)] / (2h)$$

$$D(0.1) = 0.49917$$

→ Result: 0.49917

Halved Step Size

$$D(h/2) = [f(x+h/2) - f(x-h/2)] / h$$

$$D(0.05) = 0.49979$$

→ Result: 0.49979

Richardson Extrapolation

$$D = (4/3)D(h/2) - (1/3)D(h)$$

$$D = (4/3) \times 0.49979 - (1/3) \times 0.49917 = 0.499966666666667$$

→ Result: 0.499966666666667

Second Derivative

Initial Approximation

$$D(h) = [f(x+h) - 2f(x) + f(x-h)] / h^2$$

$$D(0.1) = -0.49875$$

→ Result: -0.49875

Halved Step Size

$$D(h/2) = [f(x+h/2) - 2f(x) + f(x-h/2)] / (h/2)^2$$

$$D(0.05) = -0.49969$$

→ Result: -0.49969

Richardson Extrapolation

$$D = (16/15)D(h/2) - (1/15)D(h)$$

$$D = (16/15) \times -0.49969 - (1/15) \times -0.49875 = -0.499752666666667$$

→ Result: -0.499752666666667

6.3. Error Handling and Troubleshooting

Symptom	Possible Cause	Resolution
Derivative displays as grey.	Derivative value is near zero.	This may be correct; verify analytically.
Derivative displays as red.	Derivative is very large.	Function may have steep slope or near-singularity.
Richardson value equals CDD value	Step size too large; error terms negligible.	Use smaller h .
Result is NaN.	Invalid function evaluation.	Avoid points where function is undefined.
Truncation error estimate is large.	Step size h is too large.	Use smaller step size for better accuracy.

Source Codes

Following codes are the implementation of the selected numerical analysis algorithms. Other codes unrelated to these algorithms such as FXML visual rendering, GUI configurations, and other presentation-level classes are omitted for the sake of brevity. However, full source code may be accessed in my public

 GitHub repository: <https://github.com/kartoff-an/Numerical-Methods-Calculator>.

BaseConfigurableSolver.java

```
package com.roquero.numericalmethods.math;

import java.math.BigDecimal;
import java.math.RoundingMode;

/**
 * Base class for all solvers that need configuration settings.
 * This provides rounding capabilities without assuming expression evaluation.
 */
public abstract class BaseConfigurableSolver {

    protected SolverConfig config;

    protected BaseConfigurableSolver(SolverConfig config) {
        this.config = (config != null) ? config : SolverConfig.getDefault();
    }

    protected int getDecimalPlaces() {
        return config.getDecimalPlaces();
    }

    protected boolean isRoundEachIteration() {
        return config.isRoundEachIteration();
    }

    protected double round(double value) {
        int dp = getDecimalPlaces();
        if (dp < 0) return value;

        return BigDecimal.valueOf(value)
            .setScale(dp, RoundingMode.HALF_EVEN)
            .doubleValue();
    }

    protected double maybeRound(double value) {
        return isRoundEachIteration() ? round(value) : value;
    }

    public SolverConfig getConfig() {
        return config;
    }

    public abstract Object solve();
}
```

BaseFunctionSolver.java

```
package com.roquero.numericalmethods.math;

import net.objecthunter.exp4j.Expression;
import net.objecthunter.exp4j.ExpressionBuilder;

import java.util.function.Function;

/**
 * Base class for solvers that evaluate mathematical functions.
 * Extends BaseConfigurableSolver to include rounding capabilities.
 */
public abstract class BaseFunctionSolver extends BaseConfigurableSolver {

    protected String functionStr;
    protected boolean expressionValid = false;
    protected String validationError = null;
    protected Function<Double, Double> function;

    protected BaseFunctionSolver(String functionStr, SolverConfig config) {
        super(config);
        this.functionStr = functionStr;
        validateAndBuildExpression();
    }

    protected void validateAndBuildExpression() {
        try {
            Expression testExpr = new ExpressionBuilder(functionStr)
                .variable("x")
                .build();

            // locality validation
            double x = 0;
            double h = 0.000001;
            double[] samples = {x - 2*h, x - h, x + h, x + 2*h};

            for (double xi : samples) {
                testExpr.setVariable("x", xi).evaluate();
            }

            this.function = createFunction(functionStr);
            this.expressionValid = true;
            this.validationError = null;

        } catch (Exception e) {
            this.expressionValid = false;
            this.validationError = e.getMessage();
        }
    }

    protected Function<Double, Double> createFunction(String expression) {
        return (Double x) -> {
            try {
                Expression expr = new ExpressionBuilder(expression)
                    .variable("x")
                    .build()
                    .setVariable("x", x);
                return expr.evaluate();
            } catch (Exception e) {

```

```
        throw new RuntimeException("Error evaluating function at x = " + x, e);
    }
};

}

public double evaluateFunction(double x) {
    if (!expressionValid) {
        throw new IllegalStateException("Invalid function expression: " +
validationError);
    }
    return function.apply(x);
}

protected void checkExpressionValidity() {
    if (!expressionValid) {
        throw new IllegalStateException("Invalid function expression: " +
validationError);
    }
}

public String getValidationError() {
    return validationError;
}

public boolean isExpressionValid() {
    return expressionValid;
}
}
```

BaseIterativeSolver.java

```
package com.roquero.numericalmethods.math;

/**
 * Base class for iterative solvers that need tolerance and max iterations.
 */
public abstract class BaseIterativeSolver extends BaseFunctionSolver {

    protected double tolerance = 1e-6;
    protected int maxIterations = 100;

    protected BaseIterativeSolver(String functionStr, SolverConfig config) {
        super(functionStr, config);
    }

    public void setTolerance(double tolerance) {
        if (tolerance <= 0) {
            throw new IllegalArgumentException("Tolerance must be positive");
        }
        this.tolerance = tolerance;
    }

    public void setMaxIterations(int maxIterations) {
        if (maxIterations <= 0) {
            throw new IllegalArgumentException("Max iterations must be positive");
        }
        this.maxIterations = maxIterations;
    }
}
```

SolverConfig.java

```
package com.roquero.numericalmethods.math;

public class SolverConfig {

    private static final SolverConfig DEFAULT = new SolverConfig();

    private int decimalPlaces = 5;
    private boolean roundEachIteration = true;

    public SolverConfig() {}

    public SolverConfig(int decimalPlaces, boolean roundEachIteration) {
        this.decimalPlaces = decimalPlaces;
        this.roundEachIteration = roundEachIteration;
    }

    public static SolverConfig getDefault() {
        return DEFAULT;
    }

    public int getDecimalPlaces() {
        return decimalPlaces;
    }

    public boolean isRoundEachIteration() {
        return roundEachIteration;
    }
}
```


NewtonRaphsonSolver.java

```
package com.roquero.numericalmethods.math;

import java.util.ArrayList;
import java.util.List;

public class NewtonRaphsonSolver extends BaseIterativeSolver {

    private final double x0;

    public record Iteration(
        int iteration,
        double xn,
        double fx,
        double dfx,
        double xn1,
        double error
    ) {}

    public record Result(
        double root,
        List<Iteration> iterations,
        double convergenceTest,
        boolean converged
    ) {}

    public NewtonRaphsonSolver(String functionStr, double x0, SolverConfig config) {
        super(functionStr, config);
        this.x0 = x0;
    }

    private double firstDerivative(double x) {
        double h = Math.pow(10, -(getDecimalPlaces() + 1));
        double fxPlusH = evaluateFunction(x + h);
        double fxMinusH = evaluateFunction(x - h);
        return (fxPlusH - fxMinusH) / (2 * h);
    }

    private double secondDerivative(double x) {
        double h = Math.pow(10, -(getDecimalPlaces() + 1));
        double fxPlusH = evaluateFunction(x + h);
        double fx = evaluateFunction(x);
        double fxMinusH = evaluateFunction(x - h);
        return (fxPlusH - 2 * fx + fxMinusH) / (h * h);
    }

    @Override
    public Result solve() {
        checkExpressionValidity();

        double x = maybeRound(x0);
        List<Iteration> iterations = new ArrayList<>();

        for (int i = 1; i <= maxIterations; i++) {
            double fx;
            double dfx;

            try {
                fx = maybeRound(evaluateFunction(x));
            }
```

```

        dfx = maybeRound(firstDerivative(x));
    } catch (Exception e) {
        return new Result(x, iterations, getConvergenceTest(), false);
    }

    if (Math.abs(dfx) < 1e-12) {
        return new Result(x, iterations, getConvergenceTest(), false);
    }

    double nextX = maybeRound(x - fx / dfx);
    double error = maybeRound(Math.abs(nextX - x));

    iterations.add(new Iteration(i, x, fx, dfx, nextX, error));

    if (error <= tolerance) {
        return new Result(nextX, iterations, getConvergenceTest(), true);
    }

    x = nextX;
}

return new Result(x, iterations, getConvergenceTest(), false);
}

public double getConvergenceTest() {
    double functionValue = maybeRound(evaluateFunction(x0));
    double firstDerivative = maybeRound(firstDerivative(x0));
    double secondDerivative = maybeRound(secondDerivative(x0));

    return maybeRound(
        Math.abs(
            (functionValue * secondDerivative) / (firstDerivative *
firstDerivative)
        )
    );
}
}

```

GaussJacobiSolver.java

```
package com.roquero.numericalmethods.math;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class GaussJacobiSolver extends BaseIterativeSolver {

    private double[][] A;
    private double[] b;
    private final double[] initialValues;

    public record Iteration(
        int iteration,
        double[] xValues,
        double error
    ) {}

    public record Result(
        double[] solution,
        List<Iteration> iterations,
        boolean converged
    ) {}

    public void setTolerance(double tolerance) {
        this.tolerance = tolerance;
    }

    public void setMaxIterations(int maxIterations) {
        this.maxIterations = maxIterations;
    }

    public GaussJacobiSolver(double[][] A, double[] b, double[] initialValues, SolverConfig config) {
        super(null, config);
        this.A = A;
        this.b = b;
        this.initialValues = initialValues;
    }

    @Override
    public Result solve() {
        int size = b.length;

        if (A[0].length != initialValues.length)
            throw new IllegalArgumentException("Matrix and initial guess dimension mismatch");

        Object[] reorderedMatrix = reorderMatrix(A, b);
        A = (double[][]) reorderedMatrix[0];
        b = (double[]) reorderedMatrix[1];

        double[] x = Arrays.copyOf(initialValues, size);
        List<Iteration> iterations = new ArrayList<>();

        for (int k = 1; k <= maxIterations; k++) {
            double[] nextX = new double[size];
```

```

    for (int i = 0; i < size; i++) {
        double sum = b[i];
        for (int j = 0; j < size; j++) {
            if (i != j) {
                sum += - A[i][j] * x[j];
            }
        }

        if (Math.abs(A[i][i]) < 1e-12) {
            throw new ArithmeticException("Zero on diagonal at row " + i);
        }

        nextX[i] = maybeRound(sum / A[i][i]);
    }

    double error = 0;
    for (int i = 0; i < size; i++) {
        error = maybeRound(Math.max(error, Math.abs(nextX[i] - x[i])));
    }

    iterations.add(new Iteration(k, nextX.clone(), error));

    if (error < tolerance) {
        return new Result(nextX, iterations, true);
    }

    x = nextX;
}

return new Result(x, iterations, false);
}

public static Object[] reorderMatrix(double[][] A, double[] b) {
    int n = A.length;

    int[] perm = new int[n];
    for (int i = 0; i < n; i++) perm[i] = i;

    do {
        double[][] newA = new double[n][n];
        double[] newB = new double[n];

        for (int i = 0; i < n; i++) {
            newA[i] = Arrays.copyOf(A[perm[i]], n);
            newB[i] = b[perm[i]];
        }

        if (isDiagonallyDominant(newA)) {
            return new Object[]{newA, newB};
        }

    } while (nextPermutation(perm));

    throw new RuntimeException("No diagonally dominant arrangement found");
}

private static boolean nextPermutation(int[] arr) {
    int n = arr.length;

    int i = n - 2;
    while (i >= 0 && arr[i] >= arr[i + 1]) {
        i--;
    }
}

```

```
        if (i < 0) return false;

        int j = n - 1;
        while (arr[j] <= arr[i]) {
            j--;
        }

        swap(arr, i, j);

        reverse(arr, i + 1, n - 1);

        return true;
    }

    private static void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }

    private static void reverse(int[] arr, int left, int right) {
        while (left < right) {
            swap(arr, left++, right--);
        }
    }

    private static boolean isDiagonallyDominant(double[][] A) {
        int n = A.length;

        for (int i = 0; i < n; i++) {
            double diag = Math.abs(A[i][i]);
            double sum = 0;

            for (int j = 0; j < n; j++) {
                if (i != j) {
                    sum += Math.abs(A[i][j]);
                }
            }

            if (diag < sum) {
                return false;
            }
        }

        return true;
    }
}
```

NewtonInterpolationSolver.java

```
package com.roquero.numericalmethods.math;

import java.util.ArrayList;
import java.util.List;

public class NewtonInterpolationSolver extends BaseConfigurableSolver {

    private final double[] xValues;
    private final double[] yValues;
    private final double[][] dividedDifferences;
    private final List<Double> interpolationPoints;
    private final List<Double> interpolationResults;

    public record DividedDifferenceRow(
        int level,
        double[] values
    ) {}

    public record Result(
        double[] xValues,
        double[] yValues,
        double[][] dividedDifferencesTable,
        List<Double> interpolationPoints,
        List<Double> interpolationResults,
        String polynomial
    ) {}

    public NewtonInterpolationSolver(double[] xValues, double[] yValues, SolverConfig config)
    {
        super(config);

        if (xValues.length != yValues.length) {
            throw new IllegalArgumentException("X and Y arrays must have the same length");
        }
        if (xValues.length < 2) {
            throw new IllegalArgumentException("At least 2 points are required for interpolation");
        }

        this.xValues = xValues.clone();
        this.yValues = yValues.clone();
        this.dividedDifferences = new double[xValues.length][xValues.length];
        this.interpolationPoints = new ArrayList<>();
        this.interpolationResults = new ArrayList<>();

        calculateDividedDifferences();
    }

    private void calculateDividedDifferences() {
        int n = xValues.length;

        for (int i = 0; i < n; i++) {
            dividedDifferences[i][0] = yValues[i];
        }

        for (int j = 1; j < n; j++) {
            for (int i = 0; i < n - j; i++) {
                dividedDifferences[i][j] = maybeRound((dividedDifferences[i + 1][j - 1] -
```

```

dividedDifferences[i][j - 1])
    / (xValues[i + j] - xValues[i]));
    }
}

public double evaluate(double x) {
    int n = xValues.length;
    double result = maybeRound(dividedDifferences[0][0]);
    double product = 1.0;

    for (int i = 1; i < n; i++) {
        product = maybeRound(product * (x - xValues[i - 1]));
        result = maybeRound(result + dividedDifferences[0][i] * product);
    }

    return result;
}

public double[] interpolate(double[] points) {
    double[] results = new double[points.length];
    for (int i = 0; i < points.length; i++) {
        results[i] = maybeRound(evaluate(points[i]));
        interpolationPoints.add(maybeRound(points[i]));
        interpolationResults.add(maybeRound(results[i]));
    }
    return results;
}

public void clearInterpolationPoints() {
    interpolationPoints.clear();
    interpolationResults.clear();
}

public String getPolynomialString() {
    int n = xValues.length;
    StringBuilder polynomial = new StringBuilder();

    polynomial.append(dividedDifferences[0][0]);

    for (int i = 1; i < n; i++) {
        StringBuilder term = new StringBuilder();
        term.append(String.format(" + (%s)", dividedDifferences[0][i]));

        for (int j = 0; j < i; j++) {
            if (xValues[j] >= 0) {
                term.append(String.format("(x - %s)", xValues[j]));
            } else {
                term.append(String.format("(x + %s)", Math.abs(xValues[j])));
            }
        }

        polynomial.append(term);
    }

    return "P(x) = " + polynomial;
}

public String getSimplifiedPolynomialString() {
    int n = xValues.length;
    double[] coefficients = getPolynomialCoefficients();
    StringBuilder polynomial = new StringBuilder();

```

```

for (int i = n - 1; i >= 0; i--) {
    if (Math.abs(coefficients[i]) < 1e-10) continue;

    if (!polynomial.isEmpty()) {
        polynomial.append(coefficients[i] > 0 ? " + " : " - ");
    } else if (coefficients[i] < 0) {
        polynomial.append("-");
    }

    double absCoef = Math.abs(coefficients[i]);

    if (i == 0) {
        polynomial.append(absCoef);
    } else if (i == 1) {
        if (Math.abs(absCoef - 1.0) < 1e-10) {
            polynomial.append("x");
        } else {
            polynomial.append(absCoef)
                .append(" x");
        }
    } else {
        if (Math.abs(absCoef - 1.0) < 1e-10) {
            polynomial.append(String.format("x^%d", i));
        } else {
            polynomial.append(absCoef)
                .append(String.format(" x^%d", i));
        }
    }
}

return "P(x) = " + (polynomial.isEmpty() ? "0" : polynomial.toString());
}

public double[] getPolynomialCoefficients() {
    int n = xValues.length;
    double[] coefficients = new double[n];

    coefficients[0] = dividedDifferences[0][0];

    for (int i = 1; i < n; i++) {
        double[] product = new double[i + 1];
        product[0] = 1.0;

        for (int j = 0; j < i; j++) {
            double[] newProduct = new double[j + 2];
            for (int k = 0; k <= j; k++) {
                newProduct[k] = maybeRound(newProduct[k] + product[k] * (-xValues[j]));
                newProduct[k + 1] = maybeRound(newProduct[k + 1] + product[k]);
            }
            product = newProduct;
        }

        for (int j = 0; j <= i; j++) {
            coefficients[j] = maybeRound(coefficients[j] + dividedDifferences[0][i] *
product[j]);
        }
    }

    return coefficients;
}

public double[][] getDividedDifferencesTable() {
    return dividedDifferences.clone();
}

```



```
    }

    public Result getResult() {
        return new Result(
            xValues.clone(),
            yValues.clone(),
            dividedDifferences.clone(),
            new ArrayList<>(interpolationPoints),
            new ArrayList<>(interpolationResults),
            getSimplifiedPolynomialString()
        );
    }

    @Override
    public Object solve() {
        return getResult();
    }

    public List<Double> getInterpolationPoints() { return new
ArrayList<>(interpolationPoints); }
    public List<Double> getInterpolationResults() { return new
ArrayList<>(interpolationResults); }
}
```

SimpsonSolver.java

```
package com.roquero.numericalmethods.math;

import java.util.ArrayList;
import java.util.List;

public class SimpsonSolver extends BaseIterativeSolver {

    private final double a;
    private final double b;
    private int n;
    private double simpsonSum = 0.0;

    public record Segment(
        int index,
        double x,
        double fx,
        double coefficient
    ) {}

    public record Result(
        double integral,
        double approximateError,
        int segments,
        List<Segment> calculationDetails,
        double simpsonSum,
        boolean converged
    ) {}

    private record SimpsonCalculation(
        double integral,
        List<Segment> segments
    ) {}

    public SimpsonSolver(String functionStr, double a, double b, int n, SolverConfig config)
    {
        super(functionStr, config);
        this.a = a;
        this.b = b;
        this.n = n;
    }

    @Override
    public Result solve() {
        return solveWithTolerance(tolerance, maxIterations);
    }

    public Result solveWithTolerance(double tolerance, int maxIterations) {
        checkExpressionValidity();
        validateEven(n);

        SimpsonCalculation calculation = calculateSimpson(n);
        double error = calculateApproximateError(tolerance, maxIterations);

        return new Result(
            calculation.integral,
            error,
            n,
            calculation.segments,
        );
    }
}
```

```

        simpsonSum,
        error < tolerance);
    }

    public Result solveWithFixedSegments() {
        checkExpressionValidity();
        validateEven(n);

        SimpsonCalculation calculation = calculateSimpson(n);
        return new Result(
            calculation.integral,
            0,
            n,
            calculation.segments,
            simpsonSum,
            true);
    }

    private SimpsonCalculation calculateSimpson(int segments) {
        validateEven(segments);

        List<Segment> segmentList = new ArrayList<>();
        double h = maybeRound((b - a) / segments);

        for (int i = 0; i <= segments; i++) {
            double x = maybeRound(a + i * h);
            double fx = maybeRound(evaluateFunction(x));
            double coefficient = getCoefficient(i, segments);

            simpsonSum += maybeRound(coefficient * fx);
            segmentList.add(new Segment(i, x, fx, coefficient));
        }

        double integral = maybeRound((h / 3) * simpsonSum);
        return new SimpsonCalculation(integral, segmentList);
    }

    private double getCoefficient(int i, int totalSegments) {
        if (i == 0 || i == totalSegments) {
            return 1;
        } else if (i % 2 == 0) {
            return 2;
        } else {
            return 4;
        }
    }

    private double calculateComposite(int segments) {
        validateEven(segments);

        double h = maybeRound((b - a) / segments);
        double sum = 0;

        for (int i = 0; i <= segments; i++) {
            double x = maybeRound(a + i * h);
            double fx = maybeRound(evaluateFunction(x));
            sum += maybeRound(getCoefficient(i, segments) * fx);
        }

        return maybeRound((h / 3) * sum);
    }

    private double calculateApproximateError(double tolerance, int maxIterations) {

```

```
int currentN = n;
double previousResult = 0;
double currentResult = 0;

for (int iter = 0; iter < maxIterations; iter++) {
    currentResult = maybeRound(calculateComposite(currentN));

    if (iter > 0) {
        double error = maybeRound(Math.abs(currentResult - previousResult));
        if (error < tolerance) {
            return error;
        }
    }

    previousResult = currentResult;
    currentN = 2;
}

return maybeRound(Math.abs(currentResult - previousResult));
}

public double getA() { return a; }
public int getN() { return n; }
public void setN(int n) { this.n = n; }

protected void validateEven(int value) {
    if (value % 2 != 0) {
        throw new IllegalArgumentException("Number of segments" + " must be even. Current
value: " + value);
    }
}
}
```

CentralDividedDifferenceSolver.java

```
package com.roquero.numericalmethods.math;

import java.util.ArrayList;
import java.util.List;

public class CentralDividedDifferenceSolver extends BaseFunctionSolver {

    private double x;
    private final double h;

    public record DetailedDerivativeResult(
        double x,
        double h,
        double xn_minus_1,
        double xn,
        double xn_plus_1,
        double f_xn_minus_1,
        double f_xn,
        double f_xn_plus_1,
        double firstDerivative,
        double secondDerivative,
        double truncationError,
        String firstDerivativeFormula,
        String secondDerivativeFormula,
        String firstDerivativeCalculation,
        String secondDerivativeCalculation
    ) {}

    public record RichardsonResult(
        int order,
        double h,
        double derivative,
        double error
    ) {}

    public record RichardsonStep(
        String description,
        String formula,
        String calculation,
        double result
    ) {}

    public record RichardsonExtrapolationDetails(
        List<RichardsonResult> results,
        List<RichardsonStep> firstDerivativeSteps,
        List<RichardsonStep> secondDerivativeSteps
    ) {}

    public CentralDividedDifferenceSolver(String functionStr, double x, double h,
        SolverConfig config) {
        super(functionStr, config);
        this.x = x;
        this.h = h;
        validatePositive(h);
    }

    public double firstDerivative() {
        checkExpressionValidity();
```

```

        return maybeRound(evaluateFunction(x + h) - evaluateFunction(x - h)) / (2 * h);
    }

    public double firstDerivative(double hVal) {
        checkExpressionValidity();
        validatePositive(hVal);
        return maybeRound((evaluateFunction(x + hVal) - evaluateFunction(x - hVal)) / (2 *
hVal));
    }

    public double secondDerivative() {
        checkExpressionValidity();
        return maybeRound((evaluateFunction(x + h) - 2 * evaluateFunction(x) +
evaluateFunction(x - h)) / (h * h));
    }

    public double secondDerivative(double hVal) {
        validatePositive(hVal);
        return maybeRound((evaluateFunction(x + hVal)
- 2 * evaluateFunction(x)
+ evaluateFunction(x - hVal)) / (hVal * hVal));
    }

    public RichardsonExtrapolationDetails richardsonExtrapolation(int levels) {
        checkExpressionValidity();

        List<RichardsonResult> results = new ArrayList<>();
        List<RichardsonStep> firstSteps = new ArrayList<>();
        List<RichardsonStep> secondSteps = new ArrayList<>();

        RichardsonResult firstDerivResult = computeRichardsonTable(1, levels,
this::firstDerivative);
        results.add(firstDerivResult);
        generateRichardsonSteps(1, firstSteps);

        RichardsonResult secondDerivResult = computeRichardsonTable(2, levels,
this::secondDerivative);
        results.add(secondDerivResult);
        generateRichardsonSteps(2, secondSteps);

        return new RichardsonExtrapolationDetails(results, firstSteps, secondSteps);
    }

    protected void validatePositive(double value) {
        if (value <= 0) {
            throw new IllegalArgumentException("Step size must be positive. Current value: "
+ value);
        }
    }

    private RichardsonResult computeRichardsonTable(
        int order,
        int levels,
        java.util.function.Function<Double, Double> derivativeFunc
    ) {
        double[][] D = new double[levels][levels];

        for (int i = 0; i < levels; i++) {
            double hi = maybeRound(h / Math.pow(2, i));
            D[i][0] = maybeRound(derivativeFunc.apply(hi));
        }

        for (int j = 1; j < levels; j++) {

```

```

        for (int i = j; i < levels; i++) {
            D[i][j] = maybeRound(D[i][j - 1]
                + (D[i][j - 1] - D[i - 1][j - 1])
                / (Math.pow(2, 2 * j) - 1));
        }

        double best = D[levels - 1][levels - 1];
        double prevBest = D[levels - 1][levels - 2];
        double error = maybeRound(Math.abs(best - prevBest));

        return new RichardsonResult(order, maybeRound(h / Math.pow(2, levels - 1)), best,
error);
    }

    private void generateRichardsonSteps(int order, List<RichardsonStep> steps) {
        double h1 = h;
        double h2 = h / 2;

        if (order == 1) {
            double d1 = firstDerivative(h1);
            double d2 = firstDerivative(h2);
            double factor = 4.0/3.0;
            double factor2 = 1.0/3.0;
            double extrapolated = factor * d2 - factor2 * d1;

            steps.add(new RichardsonStep(
                "Initial Approximation",
                "D(h) = [f(x+h) - f(x-h)] / (2h)",
                String.format("D(%s) = %s", h1, d1),
                d1
            ));

            steps.add(new RichardsonStep(
                "Halved Step Size",
                "D(h/2) = [f(x+h/2) - f(x-h/2)] / h",
                String.format("D(%s) = %s", h2, d2),
                d2
            ));

            steps.add(new RichardsonStep(
                "Richardson Extrapolation",
                "D = (4/3)D(h/2) - (1/3)D(h)",
                String.format("D = (4/3)×%s - (1/3)×%s = %s", d2, d1, extrapolated),
                extrapolated
            ));
        } else {
            double d1 = secondDerivative(h1);
            double d2 = secondDerivative(h2);
            double factor = 16.0/15.0;
            double factor2 = 1.0/15.0;
            double extrapolated = factor * d2 - factor2 * d1;

            steps.add(new RichardsonStep(
                "Initial Approximation",
                "D(h) = [f(x+h) - 2f(x) + f(x-h)] / h²",
                String.format("D(%s) = %s", h1, d1),
                d1
            ));

            steps.add(new RichardsonStep(
                "Halved Step Size",
                "D(h/2) = [f(x+h/2) - 2f(x) + f(x-h/2)] / (h/2)²",

```

```

        String.format("D(%s) = %s", h2, d2),
        d2
    ));

    steps.add(new RichardsonStep(
        "Richardson Extrapolation",
        "D = (16/15)D(h/2) - (1/15)D(h)",
        String.format("D = (16/15)*%s - (1/15)*%s = %s", d2, d1, extrapolated),
        extrapolated
    ));
}

public double estimateTruncationError() {
    checkExpressionValidity();
    double h2 = maybeRound(h / 2);
    double d1 = firstDerivative();
    double d2 = firstDerivative(h2);
    return maybeRound(Math.abs(d2 - d1) / 3);
}

public DetailedDerivativeResult getDetailedDerivatives() {
    checkExpressionValidity();

    double xn_minus_1 = maybeRound(x - h);
    double xn = x;
    double xn_plus_1 = maybeRound(x + h);

    double f_xn_minus_1 = evaluateFunction(xn_minus_1);
    double f_xn = evaluateFunction(xn);
    double f_xn_plus_1 = evaluateFunction(xn_plus_1);

    double firstDeriv = firstDerivative();
    double secondDeriv = secondDerivative();
    double truncError = estimateTruncationError();

    String firstDerivativeFormula = "f'(x) = [f(x+h) - f(x-h)] / (2h)";
    String secondDerivativeFormula = "f''(x) = [f(x+h) - 2f(x) + f(x-h)] / h^2";

    String firstDerivativeCalculation = String.format(
        "f'(%s) = [f(%s) - f(%s)] / (2*%s) = [%s - %s] / %s = %s",
        x, xn_plus_1, xn_minus_1, h, maybeRound(f_xn_plus_1),
        maybeRound(f_xn_minus_1), 2 * h, firstDeriv
    );

    String secondDerivativeCalculation = String.format(
        "f''(%s) = [f(%s) - 2f(%s) + f(%s)] / %s^2 = [%s - 2*%s + %s] / %s = %s",
        x, xn_plus_1, xn, xn_minus_1, h, maybeRound(f_xn_plus_1), maybeRound(f_xn),
        maybeRound(f_xn_minus_1), h * h, secondDeriv
    );

    return new DetailedDerivativeResult(
        x, h,
        xn_minus_1, xn, xn_plus_1,
        f_xn_minus_1, f_xn, f_xn_plus_1,
        firstDeriv, secondDeriv, truncError,
        firstDerivativeFormula, secondDerivativeFormula,
        firstDerivativeCalculation, secondDerivativeCalculation
    );
}

public double getX() { return x; }
public double getH() { return h; }

```



```
public void setX(double x) { this.x = x; }

@Override
public Object solve() {
    return getDetailedDerivatives();
}
}
```

References

- Asseg, F. (n.d.). EXP4J - Download EXP4J. <https://www.objecthunter.net/exp4j/>
- Chapra, S. C., & Canale, R. P. (2015). *Numerical Methods for Engineers*. McGraw-Hill Education.
- Chapra, S. C. (2018). *Applied Numerical Methods with MATLAB® for engineers and scientists*. McGraw-Hill Education.
- GeeksforGeeks. (2025, August 28). *Jacobian method: Definition, types, examples*.
<https://www.geeksforgeeks.org/engineering-mathematics/jacobian-method/>